

Algoritmo de Grover em problemas computacionais

UA01 e UA02 — Roteiro hands-on — Algoritmos Quânticos

Relatório técnico

Integrantes do grupo:

Rafael Mecheseregian Razeira

Link dos códigos completos:

https://github.com/RafaelMRazeira/algoritmos_quanticos_hands_on_01_02

Julho de 2026

1. Introdução e contextualização do algoritmo de Grover

O algoritmo de Grover (GROVER, 1996) resolve o problema de **busca não estruturada**: encontrar, dentre $N = 2^n$ possibilidades, um item que satisfaz uma condição verificável, sem que exista qualquer ordenação ou índice que ajude na busca. Classicamente, não há alternativa melhor do que testar os candidatos um a um, o que exige em média $N/2$ consultas e, no pior caso, N consultas. O algoritmo de Grover realiza a mesma tarefa com aproximadamente $(\pi/4) * \sqrt{N}$ consultas ao oráculo, uma **aceleração quadrática**, que é comprovadamente ótima para este problema (NIELSEN; CHUANG, 2010).

O algoritmo opera em três etapas: (i) prepara-se uma **superposição uniforme** $|s\rangle$ de todos os N estados da base computacional, aplicando uma porta Hadamard a cada um dos n qubits; (ii) aplica-se repetidamente o **operador de Grover** $G = D \cdot O$, composto pelo oráculo O e pelo operador de difusão D ; (iii) mede-se o registrador, obtendo o estado marcado com alta probabilidade desde que o número de repetições r seja escolhido corretamente.

1.1 O papel do oráculo

O oráculo é o componente que **codifica o problema** dentro do circuito quântico. Ele é um operador unitário O que reconhece os estados-alvo e os marca invertendo a sua fase: $O|x\rangle = -|x\rangle$ se x é solução, e $O|x\rangle = |x\rangle$ caso contrário. É importante notar que o oráculo **não conhece a resposta**: ele apenas sabe *verificar* se um dado estado satisfaz a condição de busca exatamente como uma função booleana $f(x)$ que testa um candidato.

A marcação de fase, sozinha, é invisível a uma medição: os módulos das amplitudes permanecem iguais e o histograma seguiria uniforme. É aqui que entra a **amplificação de amplitude**: o operador de difusão $D = 2|s\rangle\langle s| - I$ reflete todas as amplitudes em torno da sua média. Como o oráculo tornou negativas as amplitudes dos alvos, a reflexão em torno da média **aumenta** as amplitudes dos estados marcados e **reduz** as dos demais. Geometricamente, cada aplicação de $G = D \cdot O$ é uma rotação de ângulo 2θ (com $\sin \theta = \sqrt{k/N}$) no plano gerado pelo estado uniforme e pela superposição dos alvos, aproximando progressivamente o vetor de estado da solução.

Dois aspectos do oráculo merecem destaque. Primeiro, a **reversibilidade**: toda operação quântica (exceto a medição) deve ser unitária e, portanto, reversível. Por isso, na construção do oráculo, as portas X usadas para mapear o estado-alvo em $|11\dots 1\rangle$ são aplicadas antes e **desfeitas depois** da inversão de fase o circuito 'descomputa' as transformações auxiliares para não deixar lixo que emaranhe o registrador e destrua a interferência. Segundo, o **impacto sobre a medição**: a probabilidade de sucesso após r iterações é $\sin^2((2r+1)\theta)$. Ela cresce até um máximo próximo de 1 no valor ótimo de r , mas, se o operador continuar sendo aplicado, a rotação **passa do ponto** e a probabilidade volta a cair. Medir no momento certo é, portanto, parte essencial do algoritmo.

O número ideal de aplicações do operador de Grover, para $N = 2^n$ estados e k alvos, é:

$$r = \lfloor (\pi/4) * \sqrt{(N/k)} \rfloor$$

2. Materiais, ambiente e método

Os experimentos foram executados em ambiente Linux com **1 vCPU e 4 GB de RAM**, utilizando Python 3.12, **Qiskit 2.5.0** e **Qiskit Aer 0.17.2** (simulador de vetor de estado). O oráculo e o operador de difusão foram construídos **manualmente, porta a porta** (X , H e X multicontrolado), sem uso de funções prontas como GroverOperator ou PhaseOracle, conforme exigido no roteiro. Nas execuções ideais (sem ruído), a porta MCX é executada nativamente pelo simulador; nas execuções com ruído, o circuito é transpilado para portas básicas (u , cx), às quais o modelo de ruído é associado.

O modelo de ruído adotado é depolarizante, com probabilidade de erro $p_1 = 0,1\%$ em portas de 1 qubit e $p_2 = 1\%$ em portas de 2 qubits (cx) valores da ordem de grandeza dos processadores supercondutores atuais. Todas as execuções usam semente fixa ($seed = 42$) para reprodutibilidade. Foram medidos: número de iterações, tempo de

execução (quântico simulado e clássico), histogramas de medição e taxa de sucesso (fração de shots que retornam um estado-alvo).

2.1 Cenários avaliados

Cenário	Qubits (n)	$N = 2^n$	Alvos (k)	$r = \lfloor (\pi/4)\sqrt{(N/k)} \rfloor$	Objetivo
1	2	4	1 ($ 11\rangle$)	1	Validação em baixa escala
2	16	65.536	1 ($ 111...1\rangle$)	201	Comparação com busca clássica
3	16→22	até $4,2 \cdot 10^6$	1	201→1608	Limite viável do ambiente
4	5	32	3 ($ 3\rangle, 7\rangle, 11\rangle$)	2	Múltiplos alvos ($k > 1$)

3. Códigos da implementação

O módulo principal (grover_lab.py) implementa o oráculo, o operador de difusão, o cálculo do número de iterações, a montagem do circuito completo, a execução no simulador e a busca clássica linear de referência. O código completo e executável está disponível no link informado na capa.

3.1 Módulo principal: grover_lab.py

```
# -*- coding: utf-8 -*-
"""
Hands-on UA01/UA02 - Algoritmo de Grover em problemas computacionais
Implementacao do algoritmo de Grover com oraculo e operador de difusao
construidos manualmente a partir de portas elementares (sem funcoes prontas).

Bibliotecas: Qiskit 2.x + Qiskit Aer.
"""

import math
import time

from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator

# -----
# 1. Oraculo (marcacao de fase dos estados-alvo)
# -----
def build_oracle(n_qubits: int, targets: list[int]) -> QuantumCircuit:
    """Oraculo de fase: aplica  $|x\rangle \rightarrow -|x\rangle$  para cada  $x$  em `targets`.

    Construção manual, porta a porta:
    - portas X levam o estado-alvo ao estado  $|11\dots1\rangle$ ;
    - um Z multicontrolado (H + MCX + H no ultimo qubit) inverte a fase;
    - as portas X são desfeitas (reversibilidade da operação unitária).
    """
    oracle = QuantumCircuit(n_qubits, name="Oraculo")
    for target in targets:
        bits = format(target, f"0{n_qubits}b")[::-1] # bit 0 = qubit 0
        zeros = [q for q in range(n_qubits) if bits[q] == "0"]
        # mapeia  $|alvo\rangle \rightarrow |11\dots1\rangle$ 
        for q in zeros:
            oracle.x(q)
        # Z multicontrolado sobre  $|11\dots1\rangle$  (inversão de fase)
        oracle.h(n_qubits - 1)
        oracle.mcx(list(range(n_qubits - 1)), n_qubits - 1)
        oracle.h(n_qubits - 1)
        # desfaz o mapeamento (garante unitariedade/reversibilidade)
        for q in zeros:
            oracle.x(q)
    return oracle

# -----
# 2. Operador de difusao (amplificacao de amplitude)
# -----
def build_diffusion(n_qubits: int) -> QuantumCircuit:
    """Operador de difusao  $D = 2|s\rangle\langle s| - I$ , construido manualmente.

    Reflexão em torno da média das amplitudes:
 $H^{\otimes n} \cdot X^{\otimes n} \cdot (Z \text{ multicontrolado}) \cdot X^{\otimes n} \cdot H^{\otimes n}$ 
    """
    diff = QuantumCircuit(n_qubits, name="Difusao")
    diff.h(range(n_qubits))
    diff.x(range(n_qubits))
```

```

diff.h(n_qubits - 1)
diff.mcx(list(range(n_qubits - 1)), n_qubits - 1)
diff.h(n_qubits - 1)
diff.x(range(n_qubits))
diff.h(range(n_qubits))
return diff

# -----
# 3. Numero ideal de iteracoes:  $r = \text{floor}(\pi/4 * \sqrt{N/k})$ 
# -----
def optimal_iterations(n_qubits: int, k_targets: int) -> int:
    N = 2 ** n_qubits
    return max(1, math.floor((math.pi / 4) * math.sqrt(N / k_targets)))

# -----
# 4. Montagem do circuito completo de Grover
# -----
def build_grover_circuit(n_qubits: int, targets: list[int],
                        iterations: int | None = None) -> tuple[QuantumCircuit, int]:
    if iterations is None:
        iterations = optimal_iterations(n_qubits, len(targets))
    oracle = build_oracle(n_qubits, targets)
    diffusion = build_diffusion(n_qubits)

    qc = QuantumCircuit(n_qubits, n_qubits)
    qc.h(range(n_qubits)) # superposicao uniforme |s>
    for _ in range(iterations): # r aplicacoes do operador de Grover
        qc.compose(oracle, inplace=True)
        qc.compose(diffusion, inplace=True)
    qc.measure(range(n_qubits), range(n_qubits))
    return qc, iterations

# -----
# 5. Execucao no simulador (com ou sem ruido)
# -----
def run_grover(n_qubits: int, targets: list[int], shots: int = 1024,
              noise_model=None, seed: int = 42):
    """Executa Grover e retorna dicionario com contagens e metricas."""
    qc, r = build_grover_circuit(n_qubits, targets)
    backend = AerSimulator(noise_model=noise_model, seed_simulator=seed)
    tqc = transpile(qc, backend) # mantem mcx nativo do Aer (ideal);
                                # com ruido, decompoe nas portas basicas

    t0 = time.perf_counter()
    result = backend.run(tqc, shots=shots).result()
    elapsed = time.perf_counter() - t0

    counts = result.get_counts()
    target_strs = {format(t, f"0{n_qubits}b") for t in targets}
    hits = sum(v for k2, v in counts.items() if k2 in target_strs)
    return {
        "circuit": qc, "iterations": r, "counts": counts,
        "time_s": elapsed, "shots": shots,
        "success_rate": hits / shots,
        "n_qubits": n_qubits, "targets": targets,
    }

# -----
# 6. Busca classica linear (baseline de comparacao)
# -----
def classical_linear_search(n_qubits: int, target: int):
    """Percorre todos os estados de 0 ate 2^n - 1 ate encontrar o alvo."""
    N = 2 ** n_qubits
    t0 = time.perf_counter()

```

```

found, queries = -1, 0
for x in range(N):
    queries += 1
    if x == target:          # "consulta" ao oraculo classico
        found = x
        break
elapsed = time.perf_counter() - t0
return {"found": found, "queries": queries, "time_s": elapsed, "N": N}

```

3.2 Ruído e experimentos de shots: noise_lab.py

```

# -*- coding: utf-8 -*-
"""
Experimentos de ruido e de numero de shots (Passo 3 do roteiro).
"""

import time

from qiskit import transpile
from qiskit_aer import AerSimulator
from qiskit_aer.noise import NoiseModel, depolarizing_error

from grover_lab import build_grover_circuit

def make_noise_model(p1: float = 0.001, p2: float = 0.01) -> NoiseModel:
    """Modelo de ruido depolarizante simples:
        p1 -> erro em portas de 1 qubit; p2 -> erro em portas de 2 qubits (cx).
    """
    nm = NoiseModel()
    nm.add_all_qubit_quantum_error(depolarizing_error(p1, 1), ["h", "x", "u"])
    nm.add_all_qubit_quantum_error(depolarizing_error(p2, 2), ["cx"])
    return nm

def run_with_noise(n_qubits, targets, shots, noise_model=None, seed=42,
                  keep_mcx=False):
    """Executa Grover com/sem ruido.

    keep_mcx=True mantem a porta mcx nativa do simulador (necessario para
    circuitos grandes, ja que decompor um MCX de 15 controles em portas
    basicas torna a simulacao com trajetorias de ruido inviavel).
    """
    qc, r = build_grover_circuit(n_qubits, targets)
    backend = AerSimulator(noise_model=noise_model, seed_simulator=seed)
    if keep_mcx:
        tqc = qc # sem transpilacao: o Aer executa a mcx nativamente
    else:
        tqc = transpile(qc, backend) # decompoe em portas basicas com ruido

    t0 = time.perf_counter()
    result = backend.run(
        tqc, shots=shots,
        shot_branching_enable=True,          # acelera trajetorias com ruido
        shot_branching_sampling_enable=True,
    ).result()
    elapsed = time.perf_counter() - t0

    counts = result.get_counts()
    target_strs = {format(t, f"0{n_qubits}b") for t in targets}
    hits = sum(v for k, v in counts.items() if k in target_strs)
    return {"counts": counts, "success_rate": hits / shots,
            "time_s": elapsed, "iterations": r, "shots": shots}

```

Um terceiro script (`run_experiments.py`) orquestra os quatro cenários, gera os histogramas, desenha os circuitos e consolida as métricas em um arquivo JSON, garantindo que todos os resultados apresentados a seguir sejam reproduzíveis com um único comando.

4. Resultados dos quatro cenários

4.1 Cenário 1: 2 qubits, 1 alvo (validação)

Com $n = 2$ e alvo $|11\rangle$ (estado 3), o número ideal de iterações é $r = 1$. Para este caso particular, $\theta = 30^\circ$ e uma única iteração leva a probabilidade do alvo exatamente a 100%. O resultado experimental confirma a previsão teórica: **1024/1024 shots** (100,0%) retornaram $|11\rangle$, em 2,4 ms de simulação.

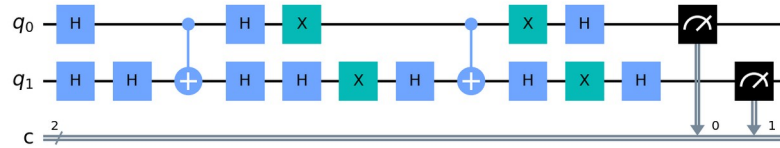


Figura 1 — Circuito completo do Cenário 1 (superposição, oráculo, difusão e medição).

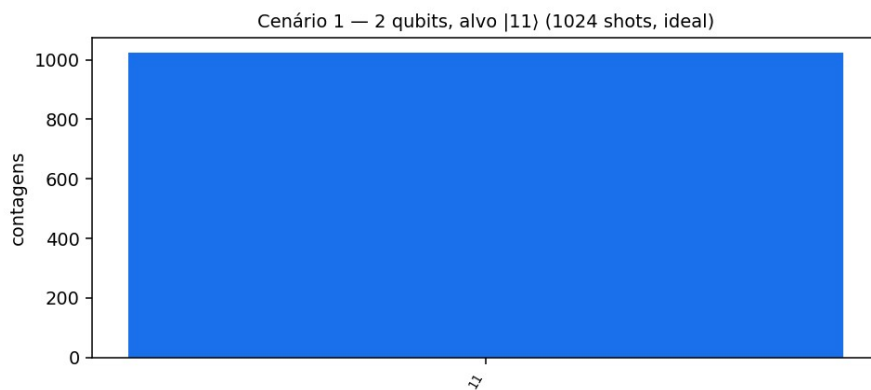


Figura 2 — Histograma do Cenário 1: todo o peso concentrado no estado-alvo $|11\rangle$.

4.2 Cenário 2: 16 qubits, 1 alvo vs. busca clássica

Com $n = 16$ ($N = 65.536$) e alvo $|111...1\rangle$ (estado 65.535), foram aplicadas $r = 201$ iterações. A versão quântica obteve **taxa de sucesso de 100,0%** em 1024 shots, com tempo de simulação de **1,53 s**. A busca clássica linear, varrendo os estados de 0 até $2^{16} - 1$, encontrou o alvo na posição 65.535 após **65.536 consultas** em apenas **2,28 ms**.

O contraste é o resultado mais instrutivo do experimento: em número de **consultas ao oráculo**, Grover usou 201 contra 65.536 da busca clássica, uma redução de $\sim 326\times$, coerente com a escala $\sqrt{N}$. Em **tempo de parede**, porém, a simulação quântica foi $\sim 680\times$ mais lenta, pois cada iteração de Grover simulada classicamente custa a manipulação de um vetor de 2^{16} amplitudes complexas. A vantagem quântica está no número de consultas, e só se converte em vantagem de tempo em hardware quântico real (ver Seção 6).

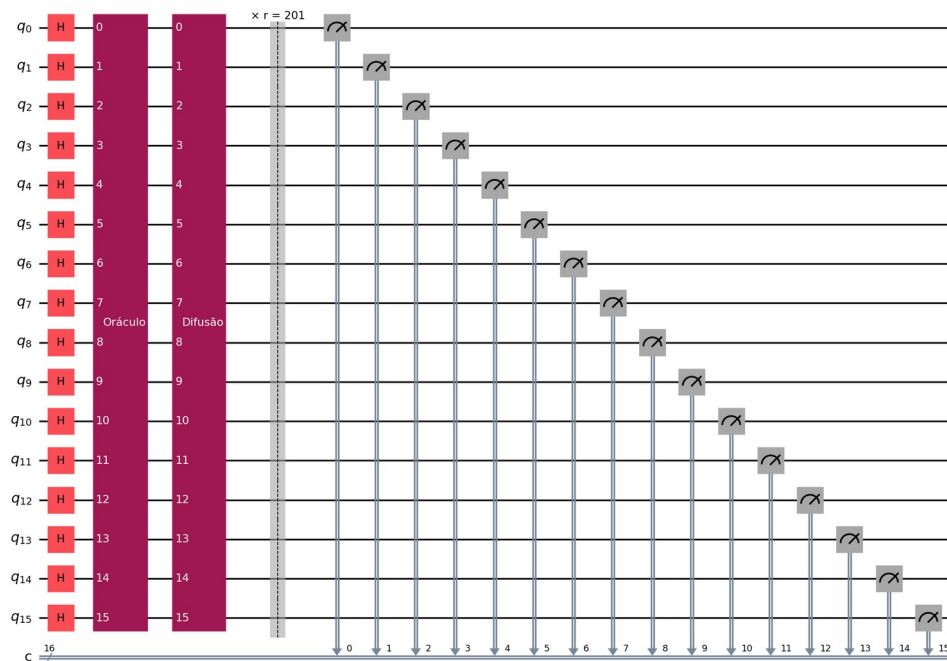


Figura 3 — Estrutura do circuito do Cenário 2 (blocos oráculo + difusão repetidos $r = 201$ vezes).

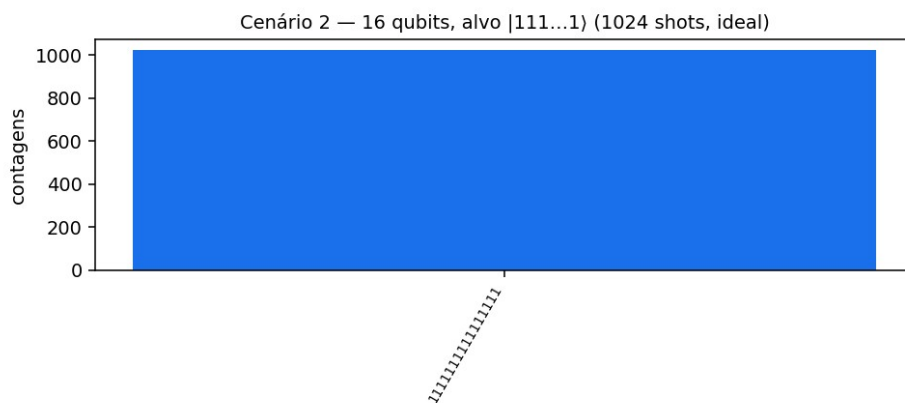


Figura 4 — Histograma do Cenário 2 (ideal): 100% das medições no estado-alvo 1111111111111111.

4.3 Cenário 3: maior número de qubits viável

n (qubits)	N	r (iterações)	Tempo de simulação	Sucesso	Situação
16	65.536	201	1,5 s	100%	ok
18	262.144	402	12,0 s	100%	ok
20	1.048.576	804	109,8 s	100%	ok
22	4.194.304	1608	—	—	abortado (>280 s sem concluir)

No ambiente utilizado (1 vCPU, 4 GB de RAM), o **limite prático foi $n = 20$ qubits** ($N \approx 1,05$ milhão de estados), executado em ~ 110 s. A tentativa com $n = 22$ foi abortada após 280 s sem conclusão, o tempo projetado, mantida a taxa de crescimento observada ($\sim 9\times$ a cada 2 qubits), seria da ordem de 15 a 20 minutos. Não houve falhas de memória até $n = 20$ (o vetor de estado de 2^{20} amplitudes ocupa ~ 16 MB), de modo que o gargalo é o **tempo de processamento**: o custo total cresce como $O(r \cdot 2^n) = O(2^{3n/2})$, pois o número de iterações e o custo de cada porta crescem simultaneamente. Este é, na prática, o retrato da dificuldade de simular computação quântica em hardware clássico.

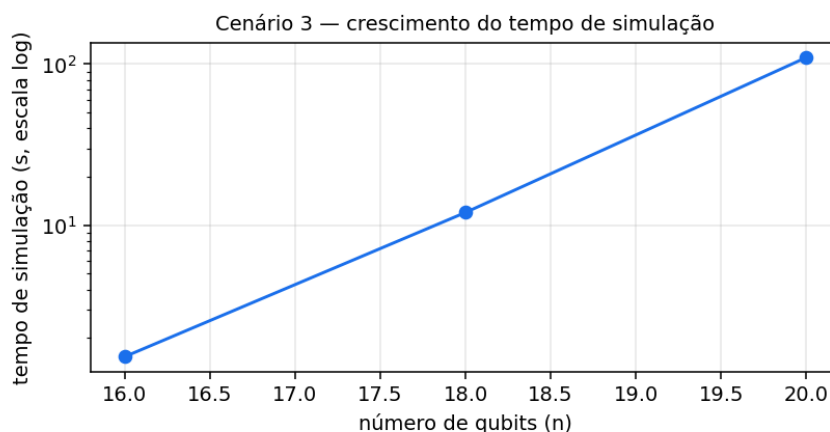


Figura 5 — Tempo de simulação em escala logarítmica: crescimento exponencial com n.

4.4 Cenário 4: 5 qubits e múltiplos alvos (k = 3)

Com $n = 5$ ($N = 32$) e três alvos (estados 3, 7 e 11), a fórmula ajustada para $k > 1$ resulta em $r = \lfloor (\pi/4) * \sqrt{(32/3)} \rfloor = 2$. A probabilidade teórica de sucesso é $\sin^2(5\theta) \approx 99,98\%$ com $\theta = \arcsen(\sqrt{(3/32)})$. O experimento confirmou: **100,0% dos 1024 shots** caíram em um dos três alvos — $|00011\rangle$: 329, $|00111\rangle$: 386, $|01011\rangle$: 309 — com distribuição aproximadamente equilibrada entre eles, como esperado, já que a amplificação atua igualmente sobre todos os estados marcados.

O oráculo para múltiplos alvos aplica, em sequência, um bloco de inversão de fase para cada alvo (X's de mapeamento, Z multicontrolado, X's de desfazimento), garantindo que **todos** os estados marcados recebam fase -1 . Vale notar que, com k alvos, o número ideal de iterações **diminui** ($\sqrt{N/k}$): mais soluções significam ângulo de rotação maior por iteração e erro r para cima degrada a fidelidade rapidamente.

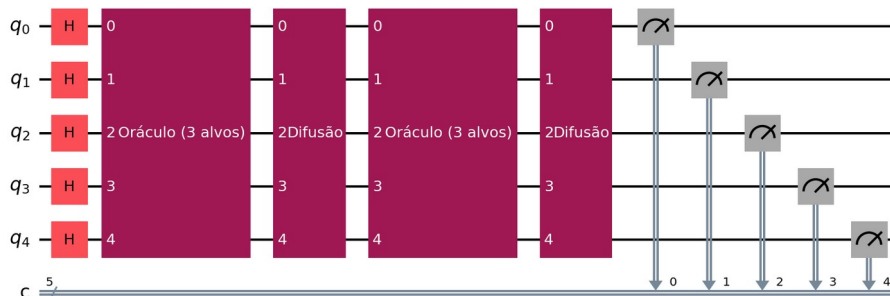


Figura 6 — Circuito do Cenário 4: duas aplicações do operador de Grover (oráculo de 3 alvos + difusão).

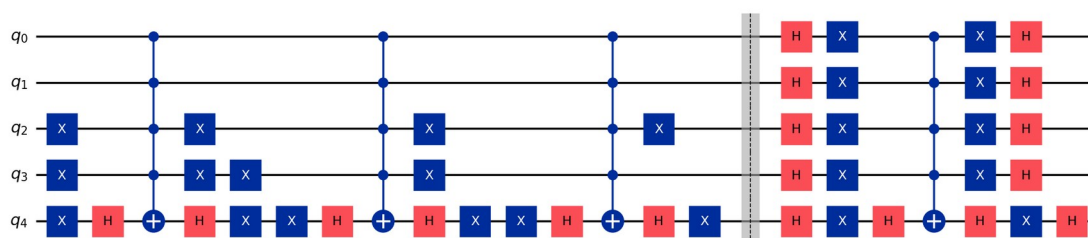


Figura 7 — Detalhe de uma iteração em portas elementares: oráculo (3 blocos de inversão de fase) e difusão.

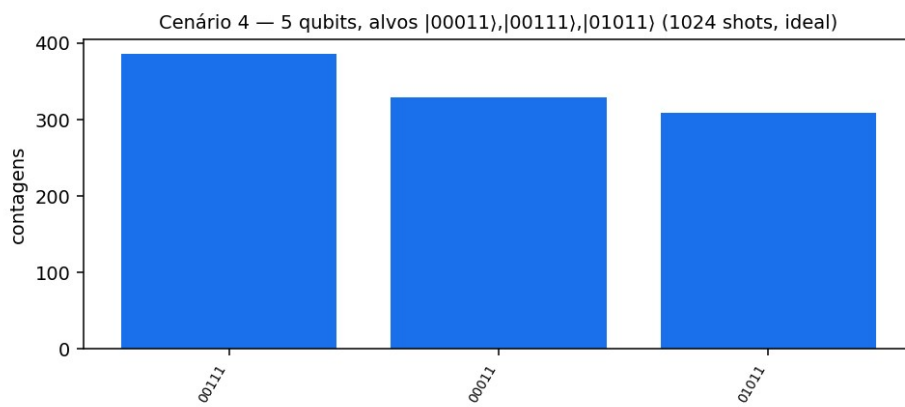


Figura 8 — Histograma do Cenário 4 (ideal): contagens concentradas nos três alvos.

5. Análise dos efeitos dos shots e do ruído

5.1 Efeito do número de shots

Os cenários 2 e 4 foram executados com 1, 100 e 1024 shots, com e sem ruído. Sem ruído, a probabilidade de sucesso teórica é $\approx 100\%$ em ambos, e o número de shots não altera o resultado apenas a **confiança estatística**: com 1 shot, um acerto isolado não permite distinguir sorte de amplificação (com ruído, o mesmo shot único do Cenário 4 acertou o alvo, sugerindo, falsamente, fidelidade perfeita); com 100 shots, o desvio padrão da frequência estimada é de ~ 5 p.p.; com 1024 shots, cai para $\sim 1,5$ p.p., e o histograma converge para a distribuição real do estado quântico. Em contrapartida, o custo cresce linearmente com os shots nas simulações com ruído (trajetórias independentes), enquanto na simulação ideal o custo é praticamente constante (o vetor de estado é calculado uma única vez e amostrado).

Cenário	Condição	1 shot	100 shots	1024 shots
4 (5 qubits, $k=3$)	ideal	100,0%	100,0%	100,0%
4 (5 qubits, $k=3$)	com ruído	100,0%	17,0%	17,4%
2 (16 qubits, $k=1$)	ideal	100,0%	100,0%	100,0%
2 (16 qubits, $k=1$)	com ruído	0,0%	0,0%	inviável*

*A simulação com ruído do circuito de 16 qubits exige uma trajetória completa por shot ($\sim 2,7$ s cada); 1024 shots demandariam ~ 45 minutos, acima do limite do ambiente. Como a taxa de sucesso já é 0% com 100 shots, o acréscimo de shots não alteraria a conclusão.

5.2 Efeito do ruído

No **Cenário 4**, a transpilação para portas básicas resulta em um circuito de profundidade 514, com 288 portas cx e 334 portas u. Com $p_2 = 1\%$ de erro por cx, a fidelidade esperada apenas pelas portas de 2 qubits é da ordem de $0,99^{288} \approx 5,6\%$, ou seja, a maioria das trajetórias sofre pelo menos um erro. O resultado experimental reflete isso: a taxa de sucesso caiu de 100% (ideal) para **17,4%** com 1024 shots. Este valor é apenas $\sim 1,9\times$ melhor que o acaso ($3/32 \approx 9,4\%$): o ruído destruiu quase toda a amplificação, espalhando as contagens por todos os 32 estados (Figura 9).

No **Cenário 2**, o efeito é ainda mais drástico: com 201 iterações e milhares de portas com erro de 0,1% cada, a probabilidade de uma trajetória atravessar o circuito sem nenhum erro é praticamente nula. A taxa de sucesso medida foi **0%** (1 e 100 shots) e o histograma é indistinguível de ruído uniforme (Figura 10), o estado-alvo sequer aparece entre os mais frequentes, pois há 65.536 estados competindo. A lição central: a profundidade do circuito de Grover cresce com $\sqrt{N}$, e **sem correção de erros a fidelidade decai exponencialmente com a profundidade**, inviabilizando instâncias grandes em hardware ruidoso (era NISQ).

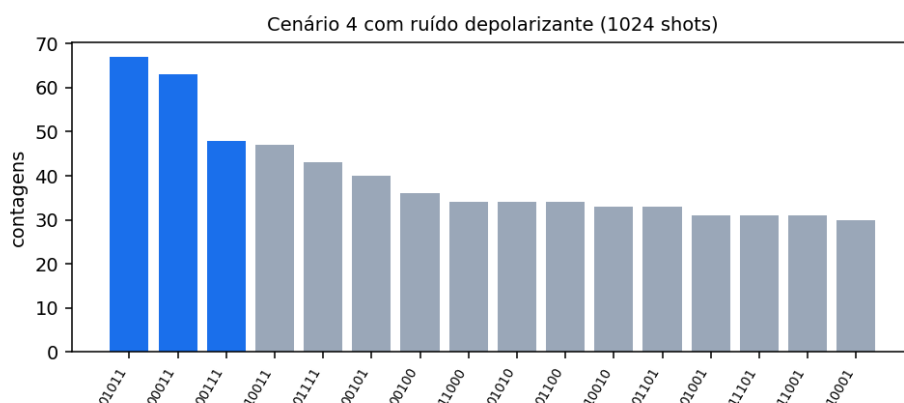


Figura 9 — Cenário 4 com ruído depolarizante: os alvos ainda lideram, mas a amplificação quase desaparece.

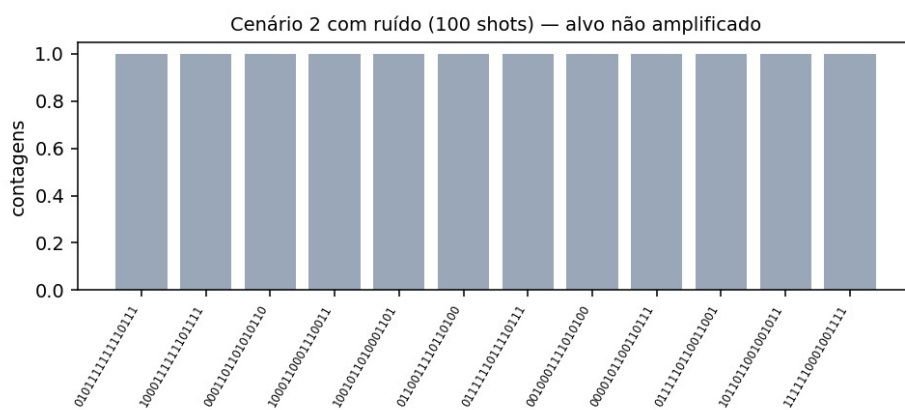


Figura 10 — Cenário 2 com ruído (100 shots): distribuição uniforme; o alvo não é amplificado.

6. Estimativa do ponto de virada: quando Grover supera a busca clássica?

A partir dos tempos medidos no Cenário 2: o tempo médio de **uma iteração de Grover** (oráculo + difusão) na simulação foi $t_{\text{iter}} \approx 7,62$ ms, enquanto o tempo médio de **uma verificação clássica** foi $t_{\text{check}} \approx 35$ ns uma razão de aproximadamente **219.369x**.

A condição para Grover vencer em tempo total é $(\pi/4) \cdot \sqrt{N} \cdot t_{\text{iter}} < (N/2) \cdot t_{\text{check}}$, isto é, $\sqrt{N} > (\pi/2) \cdot (t_{\text{iter}}/t_{\text{check}})$. Com a razão medida na simulação, o ponto de virada ficaria em $N^* \approx 1,2 \cdot 10^{11}$ ($n \approx 37$ qubits). Contudo, **na simulação clássica esse ponto jamais é alcançado**: t_{iter} não é constante cresce proporcionalmente a 2^n (Cenário 3), de modo que aumentar n afasta o alvo na mesma velocidade em que nos aproximamos dele. Em outras palavras, os dados confirmam empiricamente que **não há vantagem quântica em Grover simulado**; a vantagem é uma propriedade do hardware quântico, onde t_{iter} independe de N (a superposição é física, não armazenada em memória).

Projetando para hardware real: em processadores supercondutores atuais, uma iteração de Grover (dezenas de portas de $\sim 0,1$ – 1 μ s, considerando a decomposição do MCX) custa da ordem de $t_{\text{iter}} \approx 10^{-5}$ – 10^{-4} s, enquanto uma verificação clássica simples custa $t_{\text{check}} \approx 10^{-9}$ – 10^{-8} s (razão de 10^3 a 10^5). Com razão 10^4 , o ponto de virada seria $N^* \approx ((\pi/2) \cdot 10^4)^2 \approx 2,5 \cdot 10^8$, cerca de **28 qubits**, um tamanho modesto. O obstáculo real não é o número de qubits, mas a **fidelidade**: como mostrou a Seção 5, $r = (\pi/4) \cdot \sqrt{N} \approx 12.000$ iterações (para $n = 28$) exigiriam milhões de portas praticamente sem erro, o que só é alcançável com **correção quântica de erros** e qubits lógicos, tecnologia ainda em maturação. A estimativa realista, portanto, é que o ponto de virada prático depende menos de N e mais do advento de hardware tolerante a falhas; além disso, para problemas com estrutura (bancos indexados, por exemplo), algoritmos clássicos $O(\log N)$ continuam imbatíveis. Grover só compete em buscas genuinamente não estruturadas, como inversão de funções criptográficas e satisfação de restrições.

7. Conclusão

Os quatro cenários propostos foram implementados e executados com sucesso, com oráculo e operador de difusão construídos manualmente a partir de portas elementares. Os resultados ideais reproduziram com precisão as previsões teóricas: 100% de sucesso nos cenários 1, 2 e 4 (este último com três alvos e r ajustado para $k > 1$) e escalabilidade da simulação até 20 qubits no ambiente disponível, com crescimento exponencial do tempo, evidência direta do custo $O(2^{3n/2})$ de simular Grover classicamente.

As principais limitações encontradas foram: (i) o **custo exponencial da simulação**, que abortou o caso de 22 qubits e inviabilizou 1024 shots com ruído em 16 qubits; (ii) a **fragilidade ao ruído**: mesmo taxas de erro modestas (0,1%/1%) reduziram a taxa de sucesso de 100% para 17% no circuito raso (Cenário 4) e para 0% no circuito profundo (Cenário 2), devido ao decaimento exponencial da fidelidade com a profundidade; e (iii) o fato de a vantagem em número de consultas (201 vs. 65.536) não se traduzir em vantagem de tempo fora de hardware quântico real.

Como recomendações: o algoritmo de Grover tende a ser útil, no futuro, em contextos de busca não estruturada de grande escala, criptoanálise (busca de chaves, motivando a duplicação de tamanhos de chave simétrica), problemas de satisfação de restrições e subrotinas de amplificação de amplitude em algoritmos maiores condicionado ao desenvolvimento de computadores quânticos com correção de erros. No curto prazo (NISQ), instâncias pequenas como as deste trabalho têm valor sobretudo didático e de validação de hardware. Reproduzir este estudo em um processador quântico real, comparando com os resultados simulados aqui obtidos, é a extensão natural deste trabalho.

Referências bibliográficas

GROVER, L. K. A fast quantum mechanical algorithm for database search. **Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing (STOC '96)**, Philadelphia, 22-24 May 1996, p. 212-219.

IBM QUANTUM LEARNING. **Grover's algorithm**. In: Fundamentals of Quantum Algorithms. 2025. Disponível em: <https://learning.quantum.ibm.com/course/fundamentals-of-quantum-algorithms/grovers-algorithm>. Acesso em: 03 jul. 2026.

NIELSEN, M. A.; CHUANG, I. L. **Quantum computation and quantum information**. 2. ed. Cambridge: Cambridge University Press, 2010.

QISKIT. **Qiskit SDK documentation** (v2.x). Disponível em: <https://quantum.cloud.ibm.com/docs>. Acesso em: 03 jul. 2026.

WRIGHT, J.; TSENG, T. **Lecture 4: Grover's algorithm**. In: 15-859BB: Quantum Computation and Information Lecture Notes. Carnegie Mellon University, 21 set. 2015. Disponível em: <https://www.cs.cmu.edu/~odonnell/quantum15/lecture04.pdf>. Acesso em: 03 jul. 2026.